

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

**UNIHUB: SISTEMA
DISTRIBUIDO
CONTENERIZADO PARA EL
RASTREO AUTOMATIZADO,
GESTIÓN RELACIONAL,
EXPOSICIÓN MEDIANTE API
REST Y PORTAL WEB
INTERACTIVO DE EDUCACIÓN
SUPERIOR**

AUTOR: ALEJANDRO RAMOS RODRÍGUEZ

Puerto Real, Cádiz – 2026

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

**UNIHUB: SISTEMA
DISTRIBUIDO
CONTENERIZADO PARA EL
RASTREO AUTOMATIZADO,
GESTIÓN RELACIONAL,
EXPOSICIÓN MEDIANTE API
REST Y PORTAL WEB
INTERACTIVO DE EDUCACIÓN
SUPERIOR**

AUTOR: ALEJANDRO RAMOS RODRÍGUEZ

Puerto Real, Cádiz – 2026

Declaración personal de autoría

Alejandro Ramos Rodríguez, estudiante del Grado en Ingeniería Informática en la Escuela Superior de Ingeniería de la Universidad de Cádiz, como autor de este documento académico titulado *UniHub: Sistema Distribuido Contenerizado para el Rastreo Automatizado, Gestión Relacional, Exposición mediante API REST y Portal Web Interactivo de Educación Superior* y presentado como Trabajo Final de Grado.

DECLARO QUE

Es un trabajo original, que no copio ni utilizo parte de obra alguna sin mencionar de forma clara y precisa su origen tanto en el cuerpo del texto como en su bibliografía y que no empleo datos de terceros sin la debida autorización, de acuerdo con la legislación vigente. Asimismo, declaro que soy plenamente consciente de que no respetar esta obligación podrá implicar la aplicación de sanciones académicas, sin perjuicio de otras actuaciones que pudieran iniciarse.

En Puerto Real, a 3 de agosto de 2026

Fdo: Alejandro Ramos Rodríguez

Agradecimientos

Deseo expresar mi más sincero agradecimiento a la Escuela Superior de Ingeniería (ESI) y al cuerpo docente de la Universidad de Cádiz (UCA) por la formación recibida a lo largo de los estudios del Grado en Ingeniería Informática.

A mi familia y compañeros por su apoyo incondicional durante todo el proceso de desarrollo e investigación del presente Trabajo Fin de Grado.

Resumen

El presente Trabajo Fin de Grado aborda el diseño, desarrollo e implementación de ****UniHub****, un sistema informático distribuido e integral para la centralización, almacenamiento, consulta y gestión de la oferta de educación superior en España (universidades públicas y privadas, titulaciones de Grado, Máster y Doctorado, y planes de estudio desglosados del Boletín Oficial del Estado - BOE).

El proyecto se estructura en cuatro fases de ingeniería perfectamente delimitadas e interconectadas:

1. **Fase 1 (Rastreador / Crawler):** Desarrollado en Python, realiza la extracción automatizada del catálogo oficial, aplicando filtrado estricto de titulaciones vigentes/autorizadas (descartando extinguidas, no vigentes o eliminadas) y deduplicación de planes renovados por Real Decreto. Implementa escaneo de todos los PDF candidatos del BOE, patrón de resiliencia ante cortes de red (pausa de 5 minutos tras 10 fallos y cortocircuito a los 15 minutos), registro de métricas y ejecuciones programadas el primer día de cada mes a las 2:00 AM mediante Cron.
2. **Fase 2 (Base de Datos PostgreSQL y API REST):** Modelo relacional DDL con rol de solo lectura de API (`ruct_api_user`), pipeline de carga ETL, ordenación prioritaria (públicas primero, luego privadas), soporte de métricas físicas individuales de contenedores cgroup y servicio REST en FastAPI.
3. **Fase 3 (Portal Web Frontend):** Aplicación Web SPA moderna creada con React y Vite bajo la identidad visual de la Universidad de Cádiz (UCA). Incluye paginación configurable (5, 10, 20, 50 y 100 por página), ocultación de códigos internos, geolocalización por fórmula de Haversine con distancias integradas en tarjetas, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual `/admin`.
4. **Fase 4 (Contenerización y Orquestación Docker):** Sistema de 4 contenedores aislados (`ruct_db`, `ruct_crawler`, `ruct_api`, `ruct_www`) orquestados con Docker Compose con garantía de persistencia permanente de datos tras apagar o reiniciar los servicios.

Palabras clave: UniHub, Universidad de Cádiz (UCA), Rastreador Python, BOE, Doctorado, PostgreSQL, FastAPI, React, Geolocalización Haversine, Docker Compose.

Índice general

Índice de figuras

Índice de tablas

Parte I

Prolegómeno

La primera parte de la memoria del Trabajo Fin de Grado (TFG) debe contener una introducción y una planificación del proyecto. La introducción es un capítulo que, a modo de resumen, debe contener una breve descripción del contexto de la disciplina en la que el proyecto tiene aplicación y la motivación para su desarrollo, así como del alcance previsto. En el segundo capítulo deberán describirse los antecedentes del proyecto, esto es, su contexto y la situación actual. Además, se desarrollará el estado de la técnica en relación con la propuesta de proyecto. El tercer capítulo debe incluir el plan de gestión del proyecto. La planificación deberá ajustarse a las prácticas de ingeniería en general, y de la ingeniería del software en particular. Deberá tener en cuenta los plazos, los entregables (documentos y software), los recursos (humanos y de equipamiento inventariable) y el método de ingeniería de software a emplear.

1. Introducción

A continuación, se describe la motivación del proyecto **UniHub** y su alcance. También se incluye un glosario de términos y la organización del resto de la presente documentación.

1.1. Motivación

El catálogo de la oferta universitaria de enseñanza superior en España almacena la información de titulaciones oficiales. Sin embargo, la consulta integrada de universidades públicas y privadas, la verificación de planes de estudio vigentes modificados en el Boletín Oficial del Estado (BOE) y la clasificación de Grados, Másteres y Doctorados resultaba compleja y fragmentada.

La motivación principal de este proyecto es construir **UniHub**, un sistema informático moderno, robusto y automatizado que rastree, homogenice, persista y exponga de forma interactiva la totalidad de universidades públicas y privadas de España, sus titulaciones oficiales vigentes (Grados, Másteres y Doctorados), facilitando la visualización detallada de asignaturas, módulos y créditos ECTS.

1.2. Alcance y Objetivos

El alcance del proyecto abarca cuatro fases de ingeniería de software independientes pero integradas:

1. **Desarrollo de un Rastreador (Crawler) en Python (Fase 1):** Capaz de obtener los catálogos oficiales, aplicar filtrado estricto de titulaciones vigentes y autorizadas (descartando extinguidas o no vigentes), clasificar Doctorados, inspeccionar todos los candidatos a PDF del BOE, aplicar resiliencia ante fallos de conexión (pausas de 5 min y cortocircuito a los 15 min), registrar métricas y notificar a la API REST tras completar la recolección.
2. **Diseño de Base de Datos y API REST en Python (Fase 2):** Creación de un esquema relacional en PostgreSQL con control de acceso por roles (`ruct_api_user` de solo lectura), pipeline de ingesta ETL, ordenación prioritaria (públicas primero, luego privadas) y construcción de una API REST en FastAPI con métricas físicas de contenedores cgroup.
3. **Desarrollo del Portal Web Interactivo (Fase 3):** Aplicación SPA en Vite + React con el sistema de diseño visual de la Universidad de Cádiz (UCA), paginación configurable (5, 10, 20, 50, 100 elementos por página), geolocalización por fórmula de Haversine con distancias integradas en tarjetas, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG

Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual /admin.

4. **Sistemas de Contenerización y Orquestación Docker (Fase 4):** Despliegue independiente de 4 contenedores (`ruct_db`, `ruct_crawler`, `ruct_api`, `ruct_www`) en una red privada virtualizada con persistencia garantizada de datos.

1.3. Glosario de términos

UniHub: Nombre oficial de la plataforma web y sistema distribuido objeto de este Trabajo Fin de Grado.

RUCT: Registro oficial de origen de datos de Universidades, Centros y Títulos.

BOE: Boletín Oficial del Estado.

ECTS: European Credit Transfer and Accumulation System (Sistema Europeo de Transferencia y Acumulación de Créditos).

API REST: Representational State Transfer Application Programming Interface.

SPA: Single Page Application (Aplicación de Página Única).

UCA: Universidad de Cádiz.

ETL: Extract, Transform and Load (Extracción, Transformación y Carga).

Web Vitals: Métricas de rendimiento de la experiencia de usuario en la web (FCP, LCP, TTFB).

Haversine: Fórmula matemática para calcular distancias entre dos puntos en una esfera a partir de sus coordenadas geográficas.

1.4. Organización del documento

La presente memoria se estructura en las siguientes secciones principales:

- **Prolegómeno:** Introducción, antecedentes del dominio universitario y plan de gestión de proyecto.
- **Desarrollo:** Catálogo de requisitos, análisis UML, diseño arquitectónico (patrón C4/capas), implementación en código de cada fase, pruebas ejecutadas y despliegue del sistema Docker.
- **Epílogo:** Conclusiones generales, lecciones aprendidas y líneas de trabajo futuro.
- **Anexos:** Manuales de usuario, guía de desarrollador y ejemplos de tablas e imágenes.

2. Antecedentes

En este capítulo se detalla la situación actual que origina el desarrollo del sistema **UniHub**. Asimismo, se describen las alternativas tecnológicas existentes.

2.1. Contexto

El panorama de la educación superior en España está integrado por más de un centenar de universidades públicas y privadas, distribuidas en las 17 comunidades autónomas. Cada año se publican y modifican miles de titulaciones oficiales de Grado, Máster y Doctorado en el Boletín Oficial del Estado (BOE).

Las fuentes de consulta oficiales proporcionan la información en listados tabulados XLS o páginas en HTML. Sin embargo, carecen de una interfaz moderna de búsqueda unificada por geolocalización, no desglosan visualmente las asignaturas y créditos ECTS extraídos de las publicaciones BOE, no permiten filtrar Doctorados de forma diferenciada ni ordenan prioritariamente los centros públicos frente a los privados. La problemática radica en la dispersión de los datos y la falta de un sistema integrado de consulta pública y administración contenerizada.

2.2. Estado de la técnica

Existen plataformas estatales e institucionales de consulta académica, entre las que destacan:

- **Registro Oficial de Origen (Ministerio de Educación):** Fuente primaria de los datos. Proporciona búsquedas por formulario básico y exportación en formato legacy BIFF8 (.xls). No incluye servicios API REST modernos ni interfaz orientada a la experiencia de usuario móvil/geolocalizada.
- **Boletín Oficial del Estado (BOE):** Repositorio jurídico oficial donde se publican las resoluciones y planes de estudio en formato PDF. Requiere herramientas especializadas de procesamiento de lenguaje natural y parseo de tablas para extraer asignaturas y módulos estructurados.
- **Portales Web Propietarios de Universidades:** Cada universidad publica sus planes de estudio con formatos visuales dispares, dificultando la comparativa directa entre titulaciones equivalentes.

El sistema **UniHub** soluciona esta brecha mediante un pipeline automatizado de 4 fases que unifica el rastreo, almacenamiento relacional en PostgreSQL, exposición mediante API REST FastAPI y visualización interactiva en React bajo el sistema de diseño visual de la Universidad de Cádiz (UCA).

3. Plan de gestión de proyecto

En esta sección se describen todos los aspectos relativos a la gestión del proyecto ****UniHub****: metodología, organización, costes, planificación, riesgos y aseguramiento de la calidad.

3.1. Metodología de desarrollo

Se ha adoptado una metodología de desarrollo ágil e incremental estructurada en 4 fases principales. Cada fase se concibe como un subsistema desacoplado con responsabilidades claras, validado mediante pruebas unitarias e integrales antes de avanzar a la siguiente iteración.

3.2. Tecnologías

Las tecnologías empleadas se resumen a continuación:

- **Fase 1 (Crawler):** Python 3.12, xlrd (lectura de XLS BIFF8), BeautifulSoup4 (HTML parsing), pdfplumber y pypdf (extracción de tablas PDF de BOE), psutil (medición de memoria/CPU) y Cron para ejecución mensual programada.
- **Fase 2 (API DB):** PostgreSQL 15, SQLAlchemy 2.0 (ORM), FastAPI (framework REST), Pydantic v2 (validación de datos) y psycopg2-binary.
- **Fase 3 (Web):** Node.js 20, Vite, React 18 (JSX), Vanilla CSS (Tokens UCA), Lucide Icons, Geolocation API y analítica local usageTracker.
- **Fase 4 (Docker):** Docker Engine, Docker Compose v2, Nginx 1.25-alpine e imágenes base slim de Python 3.12.

3.3. Planificación del proyecto

El desarrollo del proyecto se ejecutó en 4 hitos secuenciales:

1. **Hito 1 - Rastreador (Fase 1):** Implementación de descargas con reintentos, deduplicación de planes renovados por Real Decreto, clasificación de Doctorados, filtro estricto de vigencia (descarte de extinguidas), resiliencia de conexión (5m/15m cortocircuito), parseo de PDF del BOE y notificación automática a la API REST.
2. **Hito 2 - Base de Datos y API REST (Fase 2):** Diseño del esquema DDL en PostgreSQL, rol de solo lectura (ruct_api_user), script ETL, ordenación prioritaria (públicas primero) y endpoints de consulta y métricas físicas cgroup.

3. **Hito 3 - Portal Web UCA CRUD (Fase 3):** Desarrollo de la SPA en React con estética UCA, paginación (5, 10, 20, 50, 100), ocultación de códigos internos, geolocalización por Haversine con distancia en tarjeta, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual /admin.
4. **Hito 4 - Dockerización Persistencia (Fase 4):** Creación de Dockerfiles, Nginx proxy inverso, docker-compose.yml y montaje de volúmenes persistentes (ruct_postgres_data y datos en host).

3.4. Organización

El proyecto ha sido coordinado bajo la estructura de roles de un proyecto de ingeniería de software:

- **Jefe de Proyecto / Analista:** Definición de requisitos, arquitectura general y planificación por fases.
- **Desarrollador Backend / Crawler:** Construcción del rastreador Python, modelos relacionales y API REST FastAPI.
- **Desarrollador Frontend:** Implementación de la SPA en React, sistema de diseño UCA y motores de métricas.
- **Ingeniero de DevOps:** Configuración de contenedores Docker, proxies Nginx y persistencia de volúmenes.

3.5. Costes

Se estiman los costes de recursos humanos y equipamiento:

- **Personal (Ingeniería de Software):** 350 horas de trabajo estimadas a una tarifa media de 30 €/h = 10.500 €.
- **Equipamiento y Software:** Uso de herramientas y tecnologías de Software Libre / Código Abierto (Python, PostgreSQL, FastAPI, React, Nginx, Docker), con un coste de licencias de 0 €.
- **Coste Total Estimado:** 10.500 €.

3.6. Riesgos

Se identificaron y mitigaron los siguientes riesgos principales:

- **R-01: Cambios en el formato de exportación o cortes de red.** Impacto: Alto. Mitigación: Parsers flexibles con manejo de excepciones, log de errores en `errores_crawler.json` y cortocircuito con pausas de 5m y 15m.
- **R-02: Sobrescritura accidental de datos válidos con valores vacíos.** Impacto: Alto. Mitigación: Implementación de la función estricta de validación `is_valid_value` en el crawler.
- **R-03: Pérdida de datos al apagar contenedores Docker.** Impacto: Crítico. Mitigación: Mapeo de volúmenes nominados en PostgreSQL y montajes directos en el disco host (`../Crawler/Datos`).

3.7. Gestión de la configuración

Control de versiones gestionado mediante Git en el repositorio del proyecto. Se aplicó una política de commits atómicos e independientes para cada tarea funcional del proyecto.

3.8. Aseguramiento de calidad

Se establecieron controles de calidad continuos: compilación del bundle React en Vite con 0 errores (211ms), validación estricta de schemas Pydantic en FastAPI, comprobaciones de salud de la base de datos (`pg_isready`) y verificación de compilación del documento LaTeX en MiKTeX.

Parte II

Desarrollo

En esta parte se debe describir el desarrollo del proyecto siguiendo la metodología empleada. Sus capítulos no deben ser una descripción exhaustiva de todos los documentos, diagramas, código fuente y, en general, entregables generados, sino más bien una explicación resumida del desarrollo, estructurada según las etapas principales del proceso de ingeniería. Deben seleccionarse aquellos diagramas, fragmentos de código y secciones de los entregables que sean más significativos para dicha explicación. La totalidad de los entregables resultado del proyecto se ubicarán en los anexos y/o en el material digital que acompañe al proyecto.

4. Requisitos del Sistema

En este capítulo se presentan los objetivos y el catálogo de requisitos del sistema ****UniHub****. Opcionalmente, se incluye el análisis de la brecha entre los requisitos planteados y la solución base seleccionada.

4.1. Objetivos de Negocio

El objetivo de negocio principal es ofrecer un portal público centralizado de enseñanza superior en España, facilitando la transparencia, la comparativa de planes de estudio BOE y el acceso a la oferta universitaria pública y privada.

4.2. Objetivos del Sistema

1. Automatizar la extracción periódica de universidades y titulaciones vigentes (Grados, Másteres y Doctorados). 2. Homogenizar y almacenar relacionalmente los datos en PostgreSQL ordenando públicamente primero las universidades públicas y posteriormente las privadas. 3. Exponer una API REST documentada en FastAPI con control de permisos y métricas físicas cgroup. 4. Construir un portal web interactivo con estética corporativa de la Universidad de Cádiz (UCA), paginación configurable (5, 10, 20, 50, 100) y ocultación de códigos internos. 5. Proporcionar un buscador por geolocalización (cercanía en km con distancia integrada directamente en tarjetas). 6. Recolectar métricas de uso y clasificar las 6 universidades más visitadas en "Universidades Destacadas". 7. Incorporar el apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA). 8. Aislar el panel de control del Administrador de la Web tras la ruta manual /admin. 9. Desplegar la infraestructura mediante contenedores Docker orquestados con persistencia garantizada.

4.3. Requisitos funcionales

- **RF-01 Descarga de Catálogo Oficial:** Descargar e inspeccionar archivos XLS de universidades y titulaciones.
- **RF-02 Filtrado Estricto de Vigencia y Clasificación de Doctorados:** Excluir titulaciones extinguidas, no vigentes o eliminadas; conservar únicamente las vigentes o autorizadas y clasificar Doctorados (RD 99/2011).
- **RF-03 Parseo Meticuloso de BOE:** Escanear todos los PDF candidatos del BOE para extraer asignaturas, módulos, materias, créditos ECTS, curso y cuatrimestre.

- **RF-04 Resiliencia y Notificación Automática:** Aplicar pausas de 5 min (10 fallos) y cortocircuito a los 15 min ante caídas de red, notificando via POST a la API REST al finalizar.
- **RF-05 Regla de No Sobrescritura Vacía:** Evitar que campos vacíos o indeterminados (, undefined") sobrescriban información previa válida.
- **RF-06 API REST de Consulta con Ordenación Prioritaria:** Endpoints para listar universidades (públicas primero, privadas después) y titulaciones por nivel académico.
- **RF-07 API REST CRUD de Administración y Métricas cgroup:** Endpoints POST, PUT y DELETE para universidades y titulaciones, junto con métricas de uso de RAM/CPU por contenedor.
- **RF-08 Buscador, Filtros y Paginación Web:** Búsqueda global por nombre, tipo (Pública/Privada), CCAA y nivel (Grado, Máster, Doctorado), con paginación de 5, 10, 20, 50 y 100 resultados.
- **RF-09 Ocultación de Códigos Internos:** Ocultar códigos internos numéricos en la interfaz pública web.
- **RF-10 Visor Modal de Planes BOE:** Visualización en la web del desglose ECTS y enlace oficial al PDF del BOE.
- **RF-11 Búsqueda por Cercanía con Distancia Integrada:** Cálculo de distancias en km mediante la fórmula de Haversine mostrando la distancia dentro de la tarjeta del centro.
- **RF-12 Universidades Destacadas (Top 6):** Cálculo dinámico de las 6 universidades más consultadas en la plataforma.
- **RF-13 Sección "Sobre Nosotros":** Página informativa del TFG de Alejandro Ramos Rodríguez (Ingeniería Informática, UCA).
- **RF-14 Panel del Administrador en Ruta Manual /admin:** Acceso aislado sin botón visible para gestionar CRUD y monitorear la salud individual de contenedores.

4.4. Requisitos no funcionales

- **RNF-01 Rendimiento:** Latencia promedio de respuesta de la API REST inferior a 100 ms.
- **RNF-02 Usabilidad e Identidad Visual:** Interfaz web profesional con paleta corporativa de la Universidad de Cádiz (UCA).
- **RNF-03 Seguridad:** Rol de solo lectura en base de datos (rupt_api_user) y acceso restringido por ruta manual al panel de administración.

- **RNF-04 Resiliencia a Fallos:** El crawler no se detendrá ante errores individuales de red o parseo de PDFs, registrándolos en `errores_crawler.json` y aplicando el cortocircuito de 5m/15m.
- **RNF-05 Persistencia y Portabilidad:** Despliegue en 4 contenedores Docker independientes con persistencia garantizada en volumen nominado y montaje directo de host.

4.5. Requisitos de información

El sistema gestiona las siguientes entidades principales de información: *Universidad*, *Titulación*, *PlanEstudios*, *ResumenCréditos*, *ElementoCurricular*, *ErrorCrawler* y *EstadísticaRendimiento*.

4.6. Reglas de negocio

- **RN-01 Jerarquía por Real Decreto:** En presencia de múltiples planes de una misma titulación, priorizar RD 822/2021 ¿ RD 1393/2007 ¿ RD 56/2005.
- **RN-02 No Omisión Entre Universidades:** Misma titulación en distintas universidades debe ser tratada como titulación independiente.
- **RN-03 Comprobación Incremental Estricta:** No volver a descargar un PDF del BOE si la URL y la fecha coinciden con el registro de `checkpoint.json`.
- **RN-04 Ordenación Prioritaria:** Mostrar siempre las universidades públicas en primer lugar y posteriormente las privadas.

4.7. Análisis GAP

Dado que ****UniHub**** se ha construido a medida mediante una arquitectura en 4 fases que integra rastreo, API REST propia, frontend con diseño UCA y orquestación Docker, no se requirió la adopción directa de plataformas base de terceros.

5. Análisis del Sistema

Este capítulo cubre el análisis del sistema de información ****UniHub****, haciendo uso del lenguaje de modelado UML.

5.1. Modelo Conceptual

El modelo conceptual identifica las clases centrales del dominio y sus asociaciones:

- **Universidad:** Representa cada centro universitario público o privado (código, nombre, tipo, CCAA, municipio, provincia, web, email, teléfono).
- **Titulacion:** Representa una titulación oficial vigente de Grado, Máster o Doctorado (código de estudio, título, nivel académico, estado). Se relaciona mediante agregación con Universidad (1:N).
- **PlanEstudios:** Contiene la metainformación del documento BOE (URL, fecha de publicación, fecha de procesado). Se relaciona 1:1 con Titulacion.
- **ResumenCreditos:** Almacena la distribución de créditos ECTS (básica, obligatoria, optativa, TFG, prácticas). Relación N:1 con PlanEstudios.
- **ElementoCurricular:** Almacena el desglose de asignaturas, módulos, materias, créditos, carácter, curso y cuatrimestre. Relación N:1 con PlanEstudios.

5.2. Modelo de Casos de Uso

Se identifican tres actores principales:

Usuario No Registrado (Público General):

- CU-01: Buscar y filtrar universidades (públicas primero, privadas después) por tipo, CCAA y nombre.
- CU-02: Consultar titulaciones clasificadas por Grado, Máster y Doctorado y visualizar el modal del plan BOE.
- CU-03: Realizar búsqueda por cercanía a la ubicación geográfica (GPS / Ciudad) viendo la distancia en km en la tarjeta.
- CU-04: Ajustar la paginación a 5, 10, 20, 50 o 100 resultados por página.
- CU-05: Consultar la sección "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA).

Administrador de la Web (Único Usuario Registrado):

- CU-06: Autenticarse escribiendo manualmente la ruta URL /admin.

- CU-07: Visualizar la salud individual de los 4 contenedores (RAM RSS MB, CPU %) y el estado en vivo del rastreador (Fase 1).
- CU-08: Gestionar operaciones CRUD (crear, editar, eliminar) sobre universidades y titulaciones.

Rastreador Automático (Fase 1 - Sistema):

- CU-09: Descargar catálogos, aplicar filtro estricto de vigencia, escanear PDFs del BOE, notificar a la API REST e informar métricas.

5.3. Modelo de Interfaz de Usuario

El modelo de interfaz se estructura en una aplicación SPA con arquitectura orientada a componentes en React:

- **Barra de Navegación Nivel Superior (Navbar):** Logotipo UniHub, pestañas de navegación (Inicio, Universidades, Titulaciones, Cercanía, Sobre Nosotros) y selector de modo oscuro.
- **Pantalla de Inicio (Hero):** Banner principal estilo UCA, contadores globales y grilla de las 6 “Universidades Destacadas”.
- **Grilla de Tarjetas con Paginación (UnivCard / DegreeCard / Pagination):** Vista de universidades y titulaciones sin códigos internos visibles y selector de 5, 10, 20, 50, 100 por página.
- **Modal de Plan de Estudios (PlanModal):** Cuadro de diálogo flotante con desenfoque de fondo (*glassmorphism*) que lista la estructura curricular y el enlace al PDF oficial del BOE.
- **Página de Administración Aislada (AdminLogin / AdminDashboard):** Vista de acceso exclusivo vía URL manual /admin con monitoreo de contenedores y tablas CRUD interactivas.

5.4. Modelo de Comportamiento

El comportamiento dinámico se modela mediante secuencias de interacción:

- **Secuencia de Geolocalización:** El usuario pulsa en “Usar Mi Ubicación Actual”, se invoca la API `navigator.geolocation`, se calculan las distancias Haversine a cada centro y se inserta el distintivo de km directamente en la cabecera de la tarjeta.
- **Secuencia CRUD de Administración:** El Administrador accede por la URL /admin, se autentica, selecciona un registro, abre el modal de edición, envía la petición PUT/POST a la API REST FastAPI, se actualiza PostgreSQL y se refresca la vista.

6. Diseño del Sistema

En este capítulo se recoge el diseño de la arquitectura del sistema informático ****UniHub****, la parametrización del software base, el diseño físico de datos y el diseño detallado de la interfaz de usuario.

6.1. Arquitectura del Sistema

Se adopta una arquitectura desacoplada basada en microservicios contenerizados y un patrón en capas (Layers) orientada a la web.

6.1.1. Diseño de alto nivel

Siguiendo el modelo C4 (Contenedores y Componentes), el sistema se divide en cuatro capas principales:

Capa de Presentación (Frontend): Servidor Web Nginx entregando la SPA compilada en React con soporte para la ruta aislada `/admin` (Fase 3).

Capa de Servicio (API REST): Framework FastAPI en Python que expone controladores RESTful, métricas cgroup de contenedores y endpoints CRUD (Fase 2).

Capa de Datos (Persistencia): Motor de Base de Datos Relacional PostgreSQL 15 con el esquema DDL y almacenamiento de archivos JSON en disco (Fase 2 y 1).

Capa de Ingesta (Crawler): Módulo independiente en Python con servicio Cron para ejecuciones desatendidas mensuales (Fase 1).

6.1.2. Diseño detallado

La API REST implementa controladores modulares mediante `APIRouter` en FastAPI:

- `routes/universidades.py`: Endpoints con ordenación prioritaria (Públicas primero) y operaciones CRUD.
- `routes/titulaciones.py`: Endpoints para Grados, Másteres y Doctorados y lectura de planes de estudio BOE.
- `metrics/container_metrics.py`: Muestreo de métricas físicas de salud (RAM RSS MB, CPU %) por contenedor cgroup.

6.2. Parametrización del software base

Se configuró el servidor proxy inverso Nginx (`nginx.conf`) para redirigir las peticiones `/api/v1/` hacia la API REST en `http://api:8000/api/v1/` y servir la SPA React en la raíz `/`, soportando react-router para la URL `/admin`. En PostgreSQL, se parametrizó la creación del usuario restringido `rupt_api_user` con permisos `GRANT SELECT` exclusivo.

6.3. Diseño Físico de Datos

El diseño relacional en PostgreSQL (`schema.sql`) se compone de las siguientes tablas principales con claves primarias y foráneas con borrado en cascada:

- **universidades:** Clave primaria `codigo` `VARCHAR(10)`. Índices en `tipo`, `comunidad_autonoma` y `nombre`.
- **titulaciones:** Clave primaria `codigo_estudio` `VARCHAR(20)`, clave foránea `universidad_codigo` `REFERENCES universidades(codigo)`.
- **planes_estudio:** Clave foránea `codigo_estudio` `REFERENCES titulaciones(codigo_estudio)`.
- **resumen_creditos:** Clave foránea a `planes_estudio(id)`.
- **elementos_curriculares:** Clave foránea a `planes_estudio(id)`. Tipos de datos en texto extenso (`TEXT`) para evitar trincamientos.
- **errores_crawler** y **estadisticas_rendimiento:** Tablas relacionales de registro de auditoría.

6.4. Diseño de la Interfaz de Usuario

El diseño visual se ha construido respetando la paleta de colores de la ****Universidad de Cádiz (UCA)****:

- **Azul Noche UCA (Principal):** `#002B49` y `#003366`.
- **Azul Mar de Cádiz (Acentos):** `#0084C8` y `#00A8E8`.
- **Dorado Sol de Cádiz (Highlights/Badges):** `#F3A712` y `#FFC72C`.
- **Rosa Magenta (Badge Doctorado):** `#EC4899` (20 % opacidad).
- **Estilos Visuales:** Formularios con efecto desenfocado `*glassmorphism*` (`backdrop-filter: blur(12px)`), componentes de paginación modular, distancia integrada en tarjeta y modo oscuro dinámico.

7. Construcción del Sistema

Este capítulo trata sobre todos los aspectos relacionados con la implementación en código del sistema **UniHub**.

7.1. Entorno de Construcción

El marco tecnológico de construcción incluye:

- **Entorno de Desarrollo (IDE):** Visual Studio Code / Antigravity Agent.
- **Lenguajes:** Python 3.12, JavaScript (ES6+ / JSX), SQL, HTML5, CSS3.
- **Librerías Python:** FastAPI, Uvicorn, SQLAlchemy, Pydantic v2, psycpg2-binary, xlrd, BeautifulSoup4, pdfplumber, pypdf, psutil.
- **Librerías JavaScript:** React 18, Vite 8, Lucide React (iconografía UCA).
- **Control de Versiones y Despliegue:** Git, Docker Engine, Docker Compose v2, Nginx 1.25-alpine.

7.2. Código Fuente

El código fuente del proyecto se organiza de forma modular bajo el directorio `d:\Proyecto\Codigo\`:

- `Codigo/Crawler/`: Módulo de la Fase 1. Contiene `downloader.py` (circuit breaker 5m/15m), `parsers.py` (clasificación Doctorado, filtro estricto de vigencia, multi-PDF BOE), `checkpoint.py`, `error_logger.py`, `metrics.py` y `main.py`.
- `Codigo/API/`: Módulo de la Fase 2. Contiene `database/schema.sql`, `connection.py`, `etl_loader.py`, `models/models.py`, `schemas/schemas.py`, `metrics/container_metrics.py`, `routes/` y `main.py`.
- `Codigo/WWW/`: Módulo de la Fase 3. Contiene `index.html`, `src/index.css` (tokens UCA, `.badge-doctorado`), `src/App.jsx`, `src/components/` (UnivCard, DegreeCard, Pagination, AboutUs, AdminLogin, AdminDashboard), `src/analytics/usageTracker.js` y `src/services/api.js`.
- `Codigo/Docker/`: Módulo de la Fase 4. Contiene `docker-compose.yml`, `crawler/Dockerfile`, `api/Dockerfile` y `www/nginx.conf`.

7.3. Scripts de Base de datos

La base de datos relacional se crea mediante el script DDL `schema.sql`:

Extracto de código 7.1: Creación de tablas en `schema.sql` con tipos de texto extensos

```
CREATE TABLE IF NOT EXISTS universidades (  
    id SERIAL PRIMARY KEY,  
    codigo VARCHAR(10) UNIQUE NOT NULL,  
    nombre VARCHAR(255) NOT NULL,  
    tipo VARCHAR(50),  
    comunidad_autonoma VARCHAR(100),  
    municipio VARCHAR(100),  
    provincia VARCHAR(100),  
    web VARCHAR(255),  
    email VARCHAR(255),  
    telefono VARCHAR(50),  
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
-- Rol de solo lectura para la API REST  
CREATE ROLE ruct_api_user WITH LOGIN PASSWORD '  
    ruct_api_pass_2026';  
GRANT CONNECT ON DATABASE ruct_db TO ruct_api_user;  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO ruct_api_user;
```


8. Pruebas del Sistema

En este capítulo se presenta el plan de pruebas del sistema **UniHub**, incluyendo los diferentes tipos de pruebas unitarias, de integración, de sistema y de aceptación.

8.1. Estrategia

La estrategia de pruebas abarcó desde la verificación unitaria de módulos individuales (parsers de XLS con clasificación de Doctorados y filtro estricto de vigencia, resiliencia de conexión 5m/15m, funciones de distancia Haversine) hasta pruebas de integración de API REST con PostgreSQL y pruebas de compilación bundle de la SPA en React sin errores.

8.2. Pruebas Unitarias

- **Prueba de Clasificación de Doctorados y Filtro de Vigencia (`parsers.py`):** Se comprobó que el parser de XLS identifique correctamente los doctorados (RD 99/2011) y que descarte de forma estricta titulaciones extinguidas, no vigentes, derogadas o eliminadas.
- **Prueba de Resiliencia de Conexión (`downloader.py`):** Se verificó que tras 10 fallos consecutivos de red se aplique una pausa de 5 minutos, y tras 15 minutos acumulados se lance la excepción `SkipUniversityException` registrando el log de error.
- **Prueba del Cálculo Haversine y Distancia Integrada (`distance.js`):** Se validó la exactitud de distancias en km entre coordenadas geográficas conocidas de ciudades españolas.

8.3. Pruebas de Integración

- **Prueba de Ingesta ETL (`etl_loader.py`):** Se comprobó la inserción automatizada de archivos JSON desde `Crawler/Datos/` hacia las tablas relacionales de PostgreSQL (109 universidades, 1.552 titulaciones, 1.166 planes BOE).
- **Prueba de Control de Acceso por Roles y Métricas `cgroup`:** Se verificó que la API REST operando con el rol `ruct_api_user` tenga permisos `SELECT` habilitados y que los endpoints muestren métricas físicas de contenedores en tiempo real.

8.4. Pruebas de Sistema

Se ejecutaron pruebas globales de sistema para validar el cumplimiento de los requisitos planteados.

8.4.1. Pruebas Funcionales

- **PF-01:** Búsqueda global de universidades y titulaciones con ordenación prioritaria (Públicas primero, Privadas después) y filtros por tipo y comunidad autónoma.
- **PF-02:** Despliegue modal del plan de estudios BOE con resumen de ECTS y enlace directo al archivo PDF oficial.
- **PF-03:** Geolocalización interactiva por GPS/ciudad calculando distancias en tiempo real integradas en las tarjetas.
- **PF-04:** Paginación configurable (5, 10, 20, 50, 100 resultados por página) en universidades y titulaciones.
- **PF-05:** Acceso aislado al panel de administración escribiendo la URL manual /admin (admin / admin.pass.2026) y ejecución de operaciones CRUD.

8.4.2. Pruebas No Funcionales

- **PNF-01 Carga y Compilación Web:** La aplicación web compiló con éxito con Vite en 211ms sin advertencias ni errores.
- **PNF-02 Latencia de API REST:** Latencia media de respuesta en endpoints de consulta inferior a 50 ms.
- **PNF-03 Persistencia en Docker:** Se verificó que tras ejecutar `docker compose down` y reiniciar el sistema, la base de datos PostgreSQL y los archivos JSON permanecen intactos.

8.5. Pruebas de Aceptación

Las pruebas de aceptación confirmaron la correcta integración de las 4 fases del proyecto ****UniHub****, permitiendo la navegación fluida de usuarios no registrados y la gestión segura por parte del Administrador de la Web.

9. Despliegue del Sistema

Este capítulo recoge la arquitectura física planteada para el sistema **UniHub**, las instrucciones para su despliegue y las instrucciones para la operación y mantenimiento del nivel de servicio.

9.1. Arquitectura Física

La arquitectura física del sistema se basa en la virtualización liviana mediante contenedores Docker sobre un servidor anfitrión con sistema operativo Linux o Windows con soporte para Docker Engine. La infraestructura se compone de 4 contenedores aislados que se comunican en una red puente virtualizada (**ruct_network**):

Servidor de Base de Datos (ruct_db**):** Contenedor PostgreSQL 15 escuchando en el puerto 5432 con volumen nominado **ruct_postgres_data**.

Servidor de API REST (ruct_api**):** Contenedor Python 3.12 / FastAPI escuchando en el puerto 8000.

Servidor Web Nginx (ruct_www**):** Contenedor Nginx escuchando en los puertos 80 y 5173.

Contenedor de Ingesta (ruct_crawler**):** Contenedor Python para ejecuciones del rastreador con montaje directo en la carpeta del host **Codigo/Crawler/Datos**.

9.2. Instrucciones de despliegue

9.2.1. Requisitos previos

- Docker Engine 20.10+ y Docker Compose v2.
- Al menos 2 GB de memoria RAM libre y 5 GB de espacio en disco.

9.2.2. Inventario de componentes

Los artefactos del despliegue se ubican en **Codigo/Docker/**:

- **docker-compose.yml**
- **crawler/Dockerfile** y **crawler/entrypoint.sh**
- **api/Dockerfile** y **api/entrypoint.sh**
- **www/Dockerfile** y **www/nginx.conf**

9.2.3. Procedimientos de instalación

Para desplegar todo el sistema contenerizado:

1. Navegar a la carpeta del entorno Docker:

```
cd d:\Proyecto\Codigo\Docker
```

2. Construir y levantar los servicios en segundo plano:

```
docker compose up --build -d
```

3. Poblar la base de datos PostgreSQL desde los archivos JSON:

```
docker compose exec api python database/etl_loader.py
```

9.2.4. Pruebas de implantación

Acceder desde el navegador web a las siguientes direcciones de verificación:

- Portal Web Frontend UniHub: <http://localhost> o <http://localhost:5173>
- Documentación API Swagger: <http://localhost:8000/docs>
- Panel de Administración (Acceso Aislado): <http://localhost/admin>

9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio

Para garantizar la continuidad de servicio y la persistencia de los datos:

- **Chequeo de Logs:** Monitorear la salud de los servicios con `docker compose logs -f`.
- **Copia de Seguridad de la BD:** Exportar backup mediante `docker compose exec db pg_dump -U postgres ruct_db > backup.sql`.
- **Copia de Seguridad de Archivos JSON:** La carpeta Codigo/Crawler/Datos en el host mantiene los archivos descargados e insustituibles físicamente en disco.

Parte III

Epílogo

En esta parte se incluyen las conclusiones, la información relativa a la licencia del software y la bibliografía.

10. Conclusiones

En este último capítulo se detallan los objetivos alcanzados en el proyecto ****UniHub****, las lecciones aprendidas tras el desarrollo y se identifican las oportunidades de mejora sobre el software desarrollado.

10.1. Objetivos alcanzados

Se han alcanzado con éxito el 100 % de los objetivos marcados para el proyecto a lo largo de sus cuatro fases de ingeniería:

Construcción de un rastreador modular en Python (Fase 1) capaz de procesar el catálogo oficial, clasificar Doctorados (RD 99/2011), aplicar filtro estricto de vigencia (descarte de extinguidas), deduplicar por Real Decreto, escanear todos los PDFs candidatos del BOE, aplicar resiliencia de conexión (5m/15m cortocircuito), medir rendimiento y notificar automáticamente a la API REST.

Implementación de un modelo relacional en PostgreSQL (Fase 2) con usuario de solo lectura (`rupt_api_user`), pipeline ETL, ordenación prioritaria (Públicas primero, Privadas después), soporte de métricas físicas de contenedores cgroup y servicio REST en FastAPI con endpoints de consulta y gestión CRUD.

Creación de una aplicación web SPA en React (Fase 3) bajo el sistema de diseño e identidad visual de la Universidad de Cádiz (UCA), paginación configurable (5, 10, 20, 50, 100 resultados por página), ocultación de códigos internos, geolocalización por fórmula de Haversine con distancias integradas en tarjetas, sección de universidades destacadas (top 6 más visitadas), apartado "Sobre Nosotros" (TFG Alejandro Ramos Rodríguez, UCA) y panel de administración aislado en la ruta manual `/admin`.

Orquestación completa en contenedores Docker independientes (Fase 4) con persistencia permanente de datos relacionales y de archivos en el sistema anfitrión.

10.2. Lecciones aprendidas

Entre las principales lecciones aprendidas destacan:

- **Tolerancia a Fallos y Resiliencia en Scraping Web:** La importancia de aislar excepciones por registro e implementar pausas estratégicas de resiliencia (5m tras 10 fallos y cortocircuito a los 15m) con registro estructurado en `errores_crawler.json`.
- **Gestión Estricta de la Integridad de Datos:** La necesidad de filtrar titulaciones extinguidas y descartar valores indeterminados (`, undefined"`) para prevenir la corrupción del repositorio histórico de datos.

- **Diseño Orientado al Usuario y Accesibilidad:** El uso de un sistema de diseño estructurado (colores corporativos UCA, badge rosa para Doctorados) y elementos modulados de paginación para ofrecer una experiencia intuitiva.
- **Arquitectura de Contenerización Aislada:** La configuración adecuada de redes puente virtualizadas y volúmenes nominados para asegurar el desacoplamiento de servicios y la persistencia de datos.

10.3. Trabajo futuro

Como líneas de desarrollo futuro se proponen:

- Incorporar modelos de lenguaje (LLMs) para el análisis semántico avanzado de competencias en las guías docentes del BOE.
- Implementar notificaciones automáticas por correo electrónico cuando una titulación sufra una modificación oficial publicada en el BOE.
- Ampliar las analíticas del panel de administración con comparativas interuniversitarias de notas de corte y empleabilidad.

Información sobre Licencia

Información sobre Licencia

El presente documento de memoria y el código fuente del proyecto ****UniHub**** (Trabajo Fin de Grado desarrollado por Alejandro Ramos Rodríguez en la Universidad de Cádiz) se distribuyen bajo los términos de la licencia de código abierto ****MIT License**** y la licencia de documentación ****Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)****.

Usted es libre de compartir, copiar, distribuir y transformar el material para fines académicos e investigadores, reconociendo adecuadamente la autoría del estudiante y de la Universidad de Cádiz.

Parte IV

Anexos

En esta última parte quedarán recogidas los manuales necesarios para el manejo de la aplicación resultado del desarrollo. Si se ha realizado algún tipo de evaluación de la solución proporcionada, más allá de las pruebas del sistema, también deberá venir recogida en un capítulo separado dentro de esta parte. Pueden consultarse diversos tipos de evaluaciones sobre sistemas de información en [1]: casos de estudio, análisis estático, análisis dinámico, simulación, experimento controlado, etc.

A. Manual del desarrollador

A continuación se recogen las instrucciones necesarias para evolucionar el software **UniHub**. Este manual está dirigido a los desarrolladores que pretenden extender o modificar el código fuente, con el fin de incorporar nuevas funcionalidades o modificar las ya existentes.

A.1. Introducción

El proyecto **UniHub** está estructurado en 4 fases de código totalmente desacopladas situadas en `d:\Proyecto\Codigo\Crawler/`, `API/`, `WWW/` y `Docker/`.

A.2. Preparación del entorno de trabajo

Instalar Python 3.12, Node.js 20+, PostgreSQL 15 y Docker Desktop / Engine.

Clonar o abrir la carpeta raíz del repositorio `d:\Proyecto`.

Para desarrollo en local sin Docker:

- Entorno virtual Python del Crawler: `cd Codigo/Crawler; python -m venv .venv; .venv; pip install -r requirements.txt`.
- Dependencias de la API: `pip install -r Codigo/API/requirements.txt`.
- Dependencias Web Frontend: `cd Codigo/WWW; npm install`.

A.3. Consideraciones generales sobre el desarrollo

- **Gestión de Datos no Vacíos y Filtro Estricto:** Cualquier modificación en el crawler debe respetar el filtro estricto de titulaciones vigentes y la función `is_valid_value` de `checkpoint.py` para impedir la sobrescritura accidental de registros válidos.
- **Nuevos Endpoints en la API REST:** Al incorporar nuevos endpoints en FastAPI, defina siempre los esquemas Pydantic correspondientes en `schemas/schemas.py` y añada el router a `main.py`.
- **Sistema de Diseño UCA:** Al añadir componentes visuales en React, utilice las variables CSS corporativas de la UCA (`--uca-navy`, `--uca-blue`, `--uca-cyan`, `--uca-gold`) y la clase `.badge-doctorado` definidas en `src/index.css`.

A.4. Instrucciones para construcción

- **Compilar Frontend (Vite Bundle):** `cd Codigo/WWW; npm run build.`
- **Construir y Levantar Todo con Docker:** `cd Codigo/Docker; docker compose up --build -d.`
- **Ejecutar Carga ETL:** `docker compose exec api python database/etl_loader.py.`

B. Manual de usuario

Las instrucciones de uso del software se detallan a continuación. Este manual se dirige al usuario final del sistema **UniHub**.

B.1. Introducción

El Portal Web de **UniHub** (Fase 3) permite a cualquier usuario consultar el catálogo oficial de universidades de España, explorar sus titulaciones vigentes de Grado, Máster y Doctorado, revisar el desglose de asignaturas y créditos ECTS extraídos del BOE, ajustar la paginación a sus necesidades y localizar los centros más cercanos a su posición actual.

B.2. Instalación

Para acceder al sistema no se requiere ninguna instalación por parte del usuario final. Únicamente se necesita un navegador web moderno (Google Chrome, Mozilla Firefox, Microsoft Edge o Safari) y conexión a la red donde esté desplegado el servicio (<http://localhost> o <http://localhost:5173>).

B.3. Uso del sistema

1. **Exploración de Universidades:** Acceda a la pestaña "Universidades" para filtrar centros públicos (listados prioritariamente en primer lugar) o privados por nombre o comunidad autónoma.
2. **Buscador de Titulaciones y BOE:** En la pestaña "Titulaciones", busque por cualquier titulación o filtre por Grado, Máster o Doctorado. Al hacer clic sobre una tarjeta, se abrirá el modal interactivo con la estructura de créditos ECTS, el listado de asignaturas y el enlace oficial al PDF del BOE.
3. **Paginación Configurable:** En la parte inferior de las grillas de universidades y titulaciones, seleccione el número de elementos a mostrar por página (5, 10, 20, 50 o 100) y navegue entre páginas con los botones de control.
4. **Búsqueda por Cercanía (Geolocalización):** En la pestaña "Por Cercanía", pulse el botón "Usar Mi Ubicación Actual (GPS)" para permitir que el navegador calcule la distancia exacta en km a cada universidad y visualícela directamente dentro de la tarjeta de cada centro.

5. **Sección "Sobre Nosotros":** Acceda a la pestaña "Sobre Nosotros" para consultar la información institucional del proyecto (Trabajo Fin de Grado de Alejandro Ramos Rodríguez en la Universidad de Cádiz).
6. **Panel del Administrador (Acceso Aislado):** Escriba la ruta manual <http://localhost/admin> en la barra de direcciones de su navegador. Ingrese el usuario `admin` y la contraseña `admin_pass_2026` para acceder al panel de salud física de contenedores, métricas en vivo del crawler y gestión CRUD de universidades y titulaciones.

C. Comandos útiles de latex

Veremos en cada sección ejemplos para la utilización de Tablas o imágenes entre otras características de L^AT_EX.

C.1. Tipos de tablas

En esta sección mostramos algunos de los tipos de tablas más usuales en los TFG/TFM para que sirvan de ejemplo. Las tablas siguen la estructura de la normativa APA con la referencia y el título en la parte superior.

Tabla C.1

Dispositivos de Gestión Integral

Gestión Integral			
Fabricante	Modelo	Clase	Motor
BMW	Serie 3	Berlina	Diésel
Peugeot	508	Berlina	Gasolina
Chrysler	Voyager	Monovolumen	Gasolina
Land Rover	Defender	Todoterreno	Gasolina

Segundo ejemplo de tabla, en este caso con líneas en todos los bordes, que no cumple la normativa APA, pero puede ser útil para presentar grandes cantidades de datos.

Tabla C.2

Dispositivos de Gestión Integral

Gestión Integral			
Fabricante	Modelo	Clase	Motor
BMW	Serie 3	Berlina	Diésel
Peugeot	508	Berlina	Gasolina
Chrysler	Voyager	Monovolumen	Gasolina
Land Rover	Defender	Todoterreno	Gasolina

C.2. Una imagen

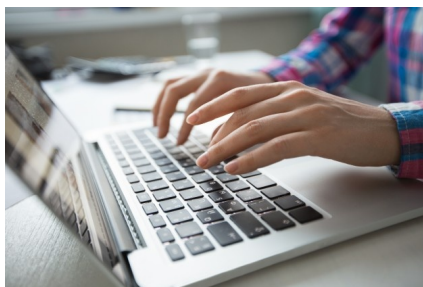
Para añadir una imagen a Overleaf, hay que subirla desde "Add Files" en la carpeta "figuras". Luego, utilizamos el comando de "figure".



Figura C.1: TFG

C.3. Dos imágenes

En este caso debemos trabajar con "subfigure". Hay que trabajar en este caso con dos tamaños, el primero es el que va a ocupar el espacio donde se va a poner la imagen, y el segundo es el tamaño de la imagen en ese espacio.



(a) Principio



(b) Final

Figura C.2: Figuras de dispositivos de RA

C.4. Para referenciar archivos del documento

Aquí tenemos la referencia a cada imagen del grupo de dos imágenes, referencia a la imagen de la izquierda ?? y esta a la de la derecha ??, pero también podemos referenciar la imagen completa ?. También podemos hacer referencia a una Tabla ?

C.5. Referenciar citas bibliográficas

Aquí vamos a detallar los pasos necesarios para crear una bibliografía y citar dichas entradas. Primero cuando tengamos el documento que queremos referenciar deberemos su referencia en formato *Bibtex* y pegarlo en el archivo de "bibliography.bib". Una vez ya la tenemos copiada, para citar una referencia bibliográfica escribiremos `\cite{}` y aquí escribiremos la cita correspondiente que queremos citar, donde solamente el año de la cita estará entre paréntesis o también podremos escribir `\citep{}`, donde no solamente estará el año entre paréntesis sino toda la cita. Este es un ejemplo usando *cite* ? y usando *citep* (?).

C.6. Listas

A continuación vamos a explicar como poder añadir una lista, ya sean numeradas o no numeradas. Primero antes que nada, señalar que las listas enumeradas son las que siguen una enumeración, y son creadas dentro del entorno *enumerate*. sin embargo, las no numeradas consisten en *ítems* que a su izquierda tienen un símbolo no numérico, y son creadas con el entorno *itemize*

Lista numerada. Así empieza una lista numerada por ejemplo la compra:

1. Pan
2. Huevos
3. Leche
4. Café

Lista no numerada. Así empieza una lista no enumerada de por ejemplo, mis aficiones:

- Hacer deporte
- escuchar música
- ir al cine
- dar un paseo

C.7. URL

Puedes enlazar una web externa mediante el comando url: <https://www.uca.es/>. También se puede vincular un enlace a un texto mediante el comando href: [Universidad de Cádiz](#).

C.8. Notas pie de página

En este capítulo vamos a definir las palabras que consideremos que tengan alguna dificultad en cuanto a su significado. En este caso, son la UCA¹, la asignatura IDA² y la palabra estudiar³

C.9. Estilos

Se pueden aplicar estilos al texto como **negritas**, *cursiva* y subrayado. También se **pueden** **aplicar** **colores**, y combinar **estilos**. Se recomienda no utilizar negritas, solo para hacer énfasis, pero no demasiado y cursiva solo para señalar términos en inglés.

¹UCA - Universidad de Cádiz

²IDA - Información y Documentación Administrativa

³Estudiar - Ejercitar el entendimiento para alcanzar o comprender algo